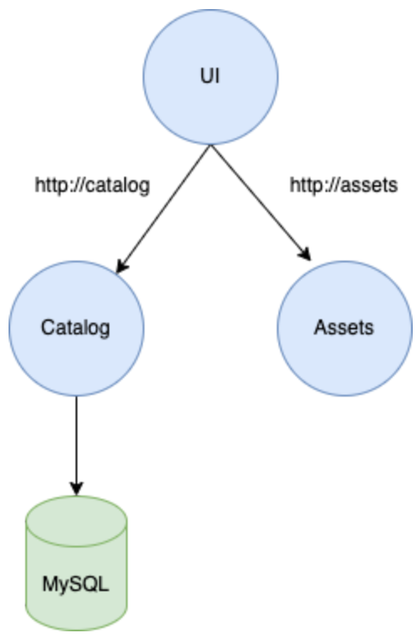
Networking

Modern applications, like our sample application, are typically built out of multiple distributed components that communicate with each other. For example, the UI component communicates via API with the Catalog component, which is linked to a persistent layer on MySQL, as shown in the architecture diagram below.



In this chapter, we will explore relevant Amazon ECS networking concepts related to Fargate. Understanding these concepts is crucial for effectively designing and deploying containerized applications on AWS.

- Amazon ECS Network Mode
- ECS Service Connect
- Amazon ECS Service Connect with TLS

Previous Next

Amazon ECS Network Mode

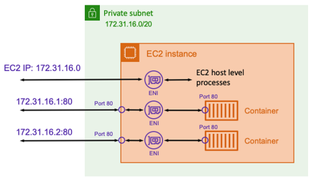
Important
You must have completed the following chapters as pre-requisites for this lab:

- Fundamentals

When running containers, it's crucial to consider the network configurations of containers running on the host. Review the documentation for more information on how to choose a network mode. In this section, we will provide an overview of the **AWSVPC mode** network configuration required for Amazon ECS on Fargate.

In **AWSVPC mode**, Amazon ECS creates and manages an Elastic Network Interface (ENI) for each task, and each task receives its own private IP address within the VPC. This configuration provides great flexibility to control communications between tasks and services at a more granular level. The AWSVPC network mode is supported for Amazon ECS tasks hosted on both Amazon EC2 and Fargate. For more information, click here.

When using Amazon ECS on Fargate, the AWSVPC network mode is required.

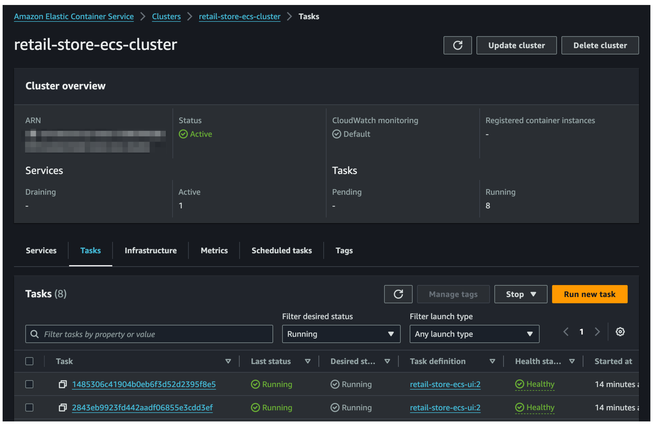


In this section, we will review the Amazon ECS network configuration.

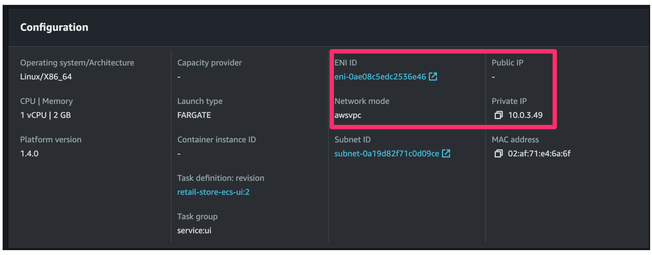
Review the Network Mode on the deployed Amazon ECS Cluster

Open the Amazon ECS console to inspect the services.

Open Amazon ECS console



From here, you can select the first running task. Scroll to the **Configuration** section to review the **Network mode**, the **ENI ID**, and the **Private IP** attached to the task:



You can access the task information programmatically by executing the following command to get the information of the task running in the `ui` service:

```
aws ecs describe-tasks \
 --cluster retail-store-ecs-cluster \
 --tasks $(aws ecs list-tasks --cluster retail-store-ecs-cluster --service ui --query 'taskArns[0]' --output text)
```


```
{
    "tasks": [
        {
            "attachments": [
                {
                    "id": "464044b3-626f-44da-86ec-fa20a064d408",
                    "type": "ElasticNetworkInterface",
                    "status": "ATTACHED",
                    "details": [
                        {
                            "name": "subnetId",
                            "value": "subnet-08c405033074ed3d"
                        },
                        {
                            "name": "networkInterfaceId",
                            "value": "eni-0a6c55131166f85c8"
                        },
                        {
                            "name": "macAddress",
                            "value": "06:39:15:1c:ae:1f"
                        },
                        {
                            "name": "privateDnsName",
                            "value": "ip-10-0-4-128.us-west-2.compute.internal"
                        },
                        {
                            "name": "privateIPv4Address",
                            "value": "10.0.4.128"
                        }
                    ]
                }
            ],
            "attributes": [
                {
                    "name": "ecs.cpu-architecture",
                    "value": "x86_64"
                }
            ],
            "availabilityZone": "us-west-2b",
            "clusterArn": "arn:aws:ecs:us-west-2:XXXXXXXXXX:cluster/retail-store-ecs-cluster",
            "connectivity": "CONNECTED",
            "connectivityAt": "2024-04-10T08:09:33.968000+00:00",
            "containers": [
                {
                    "containerArn": "arn:aws:ecs:us-west-2:XXXXXXXXXX:container/retail-store-ecs-cluster/70137dd0c1d14cf982e5a6b7446c5f54/db0fa651-1727-4215-b5a5
                    "taskArn": "arn:aws:ecs:us-west-2:XXXXXXXXXX:task/retail-store-ecs-cluster/70137dd0c1d14cf982e5a6b7446c5f54",
                    "name": "application",
                    "image": "public.ecr.aws/aws-containers/retail-store-sample-ui:0.7.0",
                    "imageDigest": "sha256:6316a3c331c39c35798f3b1303f80494526c0a879fe5a3db3b0b09a85c22aab36",
                    "runtimeId": "70137dd0c1d14cf982e5a6b7446c5f54-524788293",
                    "lastStatus": "RUNNING",
                    "networkBindings": [],
                    "networkInterfaces": [
                        {
                            "attachmentId": "464044b3-626f-44da-86ec-fa20a064d408",
                            "privateIpv4Address": "10.0.4.128"
                        }
                    ],
                    "healthStatus": "HEALTHY",
                    "cpu": "0"
                }
            ],
            "cpu": "1024",
            "createdAt": "2024-04-10T08:09:29.943000+00:00",
            "desiredStatus": "RUNNING",
            "enableExecuteCommand": false,
            "group": "service:ui",
            "healthStatus": "HEALTHY",
            "lastStatus": "RUNNING",
            "launchType": "FARGATE",
            "memory": "2048",
            "overrides": {
                "containerOverrides": [
                    {
                        "name": "application"
                    }
                ],
                "inferenceAcceleratorOverrides": []
            },
            "platformVersion": "1.4.0",
            "platformFamily": "Linux",
            "pullStartedAt": "2024-04-10T08:09:44.535000+00:00",
            "pullStoppedAt": "2024-04-10T08:09:52.398000+00:00",
            "startedAt": "2024-04-10T08:10:44.148000+00:00",
            "startedBy": "ecs-svc/8962710467093341990",
            "tags": [],
            "taskArn": "arn:aws:ecs:us-west-2:XXXXXXXXXX:task/retail-store-ecs-cluster/70137dd0c1d14cf982e5a6b7446c5f54",
            "taskDefinitionArn": "arn:aws:ecs:us-west-2:XXXXXXXXXX:task-definition/retail-store-ecs-ui:8",
            "version": 4,
            "ephemeralStorage": {
                "sizeInGiB": 20
            }
        }
    ],
    "failures": []
}
```

ECS Service Connect

Important
You must have completed the following chapters as pre-requisites for this lab:

- Fundamentals

ECS Service Connect is the recommended approach for handling service-to-service communication, offering features such as service discovery, connectivity, and traffic monitoring. With Service Connect, your applications can utilize short names and standard ports to connect to ECS services within the same cluster, across different clusters, and even across VPCs within the same AWS Region. For more detailed information, please refer to the AWS documentation.

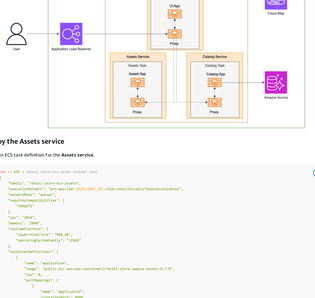
Alternative options for configuring inter-service communication within Amazon ECS Services include:

- Internal Load Balancer
- Service Discovery
- Amazon VPC Lattice

Previous Next

Enabling Service Connect

In this section, we'll enable ECS Service Connect in our cluster by deploying two additional microservices that the UI service will communicate with:



Deploy the Assets service

Create an ECS task definition for the **Assets service**.

```
cat << EOF > retail-store-ecs-asset-taskdef.json
{
  "family": "retail-store-ecs-assets",
  "executionRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskExecutionRole",
  "networkMode": "awsvpc",
  "requiresCompatibilities": [
    "FARGATE"
  ],
  "cpu": "1024",
  "memory": "2048",
  "runtimePlatform": {
    "cpuArchitecture": "X86_64",
    "operatingSystemFamily": "LINUX"
  },
  "containerDefinitions": [
    {
      "name": "application",
      "image": "public.ecr.aws/containers/retail-store-sample-assets:0.7.0",
      "cpu": 0,
      "portMappings": [
        {
          "name": "application",
          "containerPort": 8080,
          "hostPort": 8080,
          "protocol": "tcp",
          "appProtocol": "http"
        }
      ],
      "essential": true,
      "environment": [
        {
          "name": "JAVA_OPTS",
          "value": "-XX:MaxRAMPercentage=75.0 -Djava.security.egd=file:/dev/urandom"
        }
      ],
      "mountPoints": [],
      "volumesFrom": [],
      "systemControls": [],
      "healthCheck": {
        "command": [
          "CMD-SHELL",
          "curl -f http://localhost:8080/health.html || exit 1"
        ],
        "interval": 10,
        "timeout": 5,
        "retries": 3,
        "startPeriod": 60
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/retail-store-ecs-tasks",
          "awslogs-region": "${AWS_REGION}",
          "awslogs-stream-prefix": "assets-service"
        }
      }
    }
  ]
}
EOF

aws ecs register-task-definition --cli-input-json file://retail-store-ecs-asset-taskdef.json
```

Create the corresponding Assets ECS service:

```
aws ecs create-service \
  --cluster retail-store-ecs-cluster \
  --service-name assets \
  --task-definition retail-store-ecs-assets \
  --desired-count 1 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[${PRIVATE_SUBNET1}, ${PRIVATE_SUBNET2}], securityGroups=[${ASSET_SG_ID}],assignPublicIp=DISABLED}" \
  --service-connect-configuration '{
    "enabled": true,
    "namespace": "retailstore.local",
    "services": [
      {
        "portName": "application",
        "discoveryName": "assets",
        "clientAliases": [
          {
            "port": 80,
            "dnsName": "assets"
          }
        ]
      }
    ]
  }'
```

Note that when we create this service, we specify `--service-connect-configuration`, which:

1. Enables Service Connect
2. Specifies a namespace that all the services will share
3. Configures the Service Connect services that will be provided by this ECS service, including its alias and port number

For more information, refer to the AWS documentation.

Deploy the Catalog service

Create an ECS task definition for the **Catalog service**.

```
cat << EOF > retail-store-ecs-catalog-taskdef.json
{
  "family": "retail-store-ecs-catalog",
  "executionRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/catalogSysTaskExecutionRole",
  "networkMode": "awsvpc",
  "requiresCompatibilities": [
    "FARGATE"
  ],
  "cpu": "1024",
  "memory": "2048",
  "runtimePlatform": {
    "cpuArchitecture": "X86_64",
    "operatingSystemFamily": "LINUX"
  },
  "containerDefinitions": [
    {
      "name": "application",
      "image": "public.ecr.aws/aws-containers/retail-store-sample-catalog:0.7.0",
      "portMappings": [
        {
          "name": "application",
          "containerPort": 8080,
          "hostPort": 8080,
          "protocol": "tcp",
          "appProtocol": "http"
        }
      ],
      "essential": true,
      "environment": [
        {
          "name": "SB_ENV",
          "value": "catalog"
        }
      ],
      "secrets": [
        {
          "name": "DB_ENDPOINT",
          "valueFrom": "arn:aws:ssm:${AWS_REGION}:${ACCOUNT_ID}:parameter/retail-store-ecs/catalog/db-endpoint"
        },
        {
          "name": "DB_PASSWORD",
          "valueFrom": "arn:aws:secretsmanager:${AWS_REGION}:${ACCOUNT_ID}:secret:retail-store-ecs-catalog-db-password:"
        },
        {
          "name": "DB_USER",
          "valueFrom": "arn:aws:secretsmanager:${AWS_REGION}:${ACCOUNT_ID}:secret:retail-store-ecs-catalog-db-username:"
        }
      ],
      "healthCheck": {
        "command": [
          "CMD-SHELL",
          "curl -f http://localhost:8080/health || exit 1"
        ],
        "interval": 10,
        "timeout": 5,
        "retries": 3,
        "startPeriod": 60
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/retail-store-ecs-tasks",
          "awslogs-region": "${AWS_REGION}",
          "awslogs-stream-prefix": "catalog-service"
        }
      }
    }
  ]
}
EOF

aws ecs register-task-definition --cli-input-json file://retail-store-ecs-catalog-taskdef.json
```

Create the corresponding Catalog ECS service:

```
aws ecs create-service \
  --cluster retail-store-ecs-cluster \
  --service-name catalog \
  --task-definition retail-store-ecs-catalog \
  --desired-count 1 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[${PRIVATE_SUBNET1}, ${PRIVATE_SUBNET2}], securityGroups=[${CATALOG_SG_ID}],assignPublicIp=DISABLED}" \
  --service-connect-configuration '{
    "enabled": true,
    "namespace": "retailstore.local",
    "services": [
      {
        "portName": "application",
        "discoveryName": "catalog",
        "clientAliases": [
          {
            "port": 80,
            "dnsName": "catalog"
          }
        ]
      }
    ]
  }'
```

Updating the UI service

Before updating the UI service, let's wait for the new services to finish deploying (**~ 2 mins**):

```
aws ecs wait services-stable --cluster retail-store-ecs-cluster --services catalog
aws ecs wait services-stable --cluster retail-store-ecs-cluster --services assets
```

The following environment variables need to be added to the UI task definition to link the UI service with the Catalog and Asset services:

```
"environment": [
    {
        "name": "ENDPOINTS_CATALOG",
        "value": "http://catalog"
    },
    {
        "name": "ENDPOINTS_ASSETS",
        "value": "http://assets"
    }
],
```

Update the **UI task definition** with the environment variables:

```
cat << EOF > retail-store-ecs-ui-connect-taskdef.json
{
  "family": "retail-store-ecs-ui",
  "executionRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskRole",
  "networkMode": "awsvpc",
  "requiresCompatibilities": [
    "FARGATE"
  ],
  "cpu": "1024",
  "memory": "2048",
  "runtimePlatform": {
    "cpuArchitecture": "X86_64",
    "operatingSystemFamily": "LINUX"
  },
  "containerDefinitions": [
    {
      "name": "application",
      "image": "public.ecr.aws/aws-containers/retail-store-sample-ui:0.7.0",
      "portMappings": [
        {
          "name": "application",
          "containerPort": 8080,
          "hostPort": 8080,
          "protocol": "tcp",
          "appProtocol": "http"
        }
      ],
      "essential": true,
      "linuxParameters": {
        "initProcessEnabled": true
      },
      "environment": [
        {
          "name": "ENDPOINTS_CATALOG",
          "value": "http://catalog"
        },
        {
          "name": "ENDPOINTS_ASSETS",
          "value": "http://assets"
        }
      ],
      "healthCheck": {
        "command": [
          "CMD-SHELL",
          "curl -f http://localhost:8080/actuator/health || exit 1"
        ],
        "interval": 10,
        "timeout": 5,
        "retries": 3,
        "startPeriod": 60
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/retail-store-ecs-tasks",
          "awslogs-region": "${AWS_REGION}",
          "awslogs-stream-prefix": "ui-service"
        }
      }
    }
  ]
}
EOF

aws ecs register-task-definition --cli-input-json file://retail-store-ecs-ui-connect-taskdef.json
```

Now update the UI service with the new task definition (**~ 5 mins**):

```
aws ecs update-service \
  --cluster retail-store-ecs-cluster \
  --service ui \
  --task-definition retail-store-ecs-ui \
  --force-new-deployment \
  --service-connect-configuration '{
    "enabled": true,
    "namespace": "retailstore.local",
    "services": [
      {
        "portName": "application",
        "discoveryName": "ui",
        "clientAliases": [
          {
            "port": 80,
            "dnsName": "ui"
          }
        ]
      }
    ]
  }'

echo "Waiting for service to stabilize..."

aws ecs wait services-stable --cluster retail-store-ecs-cluster --services ui
```

Explore Web Application

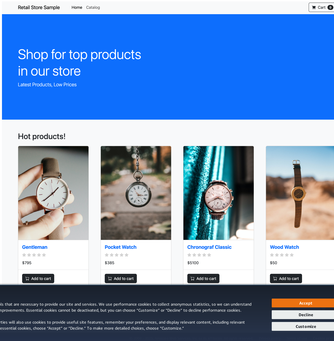
Since we've deployed not only the UI service but also the Assets and Catalog services, the application's appearance has slightly changed.

```
export RETAIL_ALB=$(aws elbv2 describe-load-balancers --name retail-store-ecs-ui \
  --query 'LoadBalancers[0].DNSName' --output text)

echo http://${RETAIL_ALB} ; echo
```

Paste the URL into a web browser to access the application.

Now that we deployed `assets` and `catalog` services, the web application will also display the product images; like in the screenshot below:

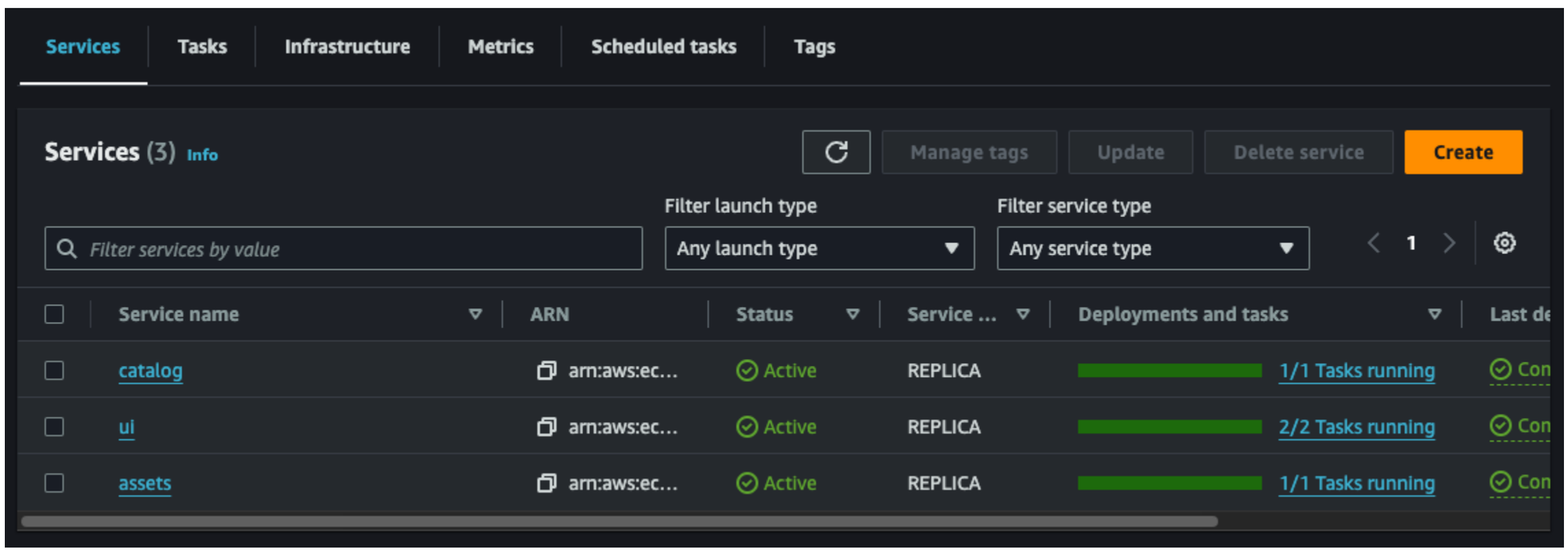


Examining the services

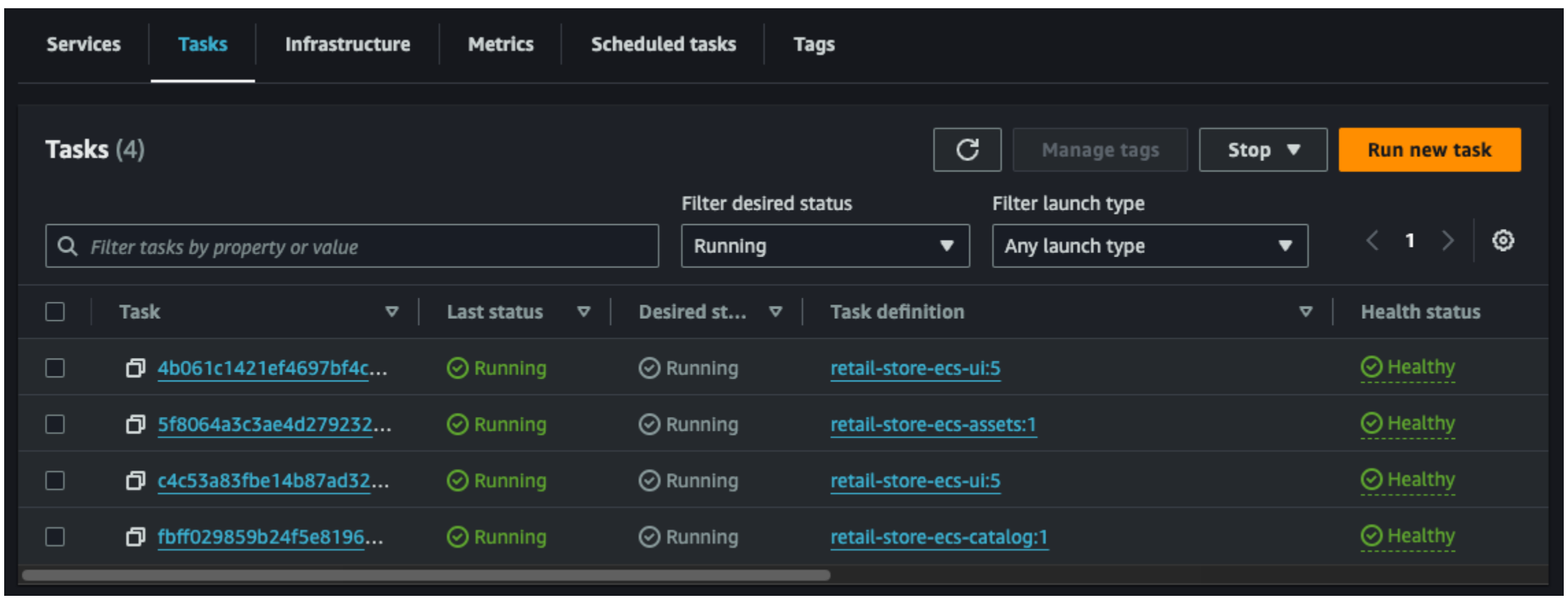
Open the Amazon ECS console to inspect the services.

Open Amazon ECS console

You'll now see three services running in the cluster:

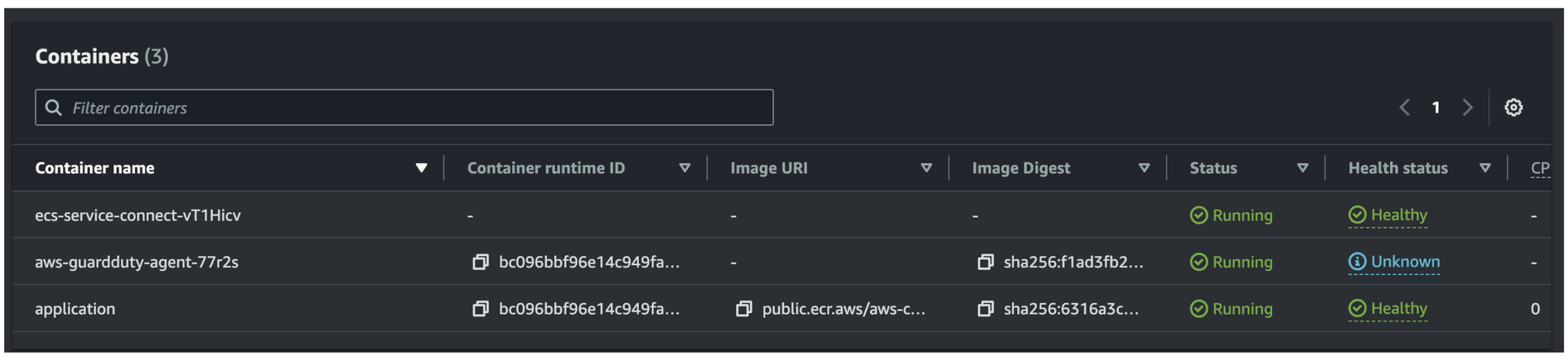


Navigating to the **Tasks** tab will show multiple tasks running:



Select one of the running tasks and scroll down to the **Containers** section. In the screen below, you can see the `application` and the `ecs-service-connect` containers. More information can be found here.

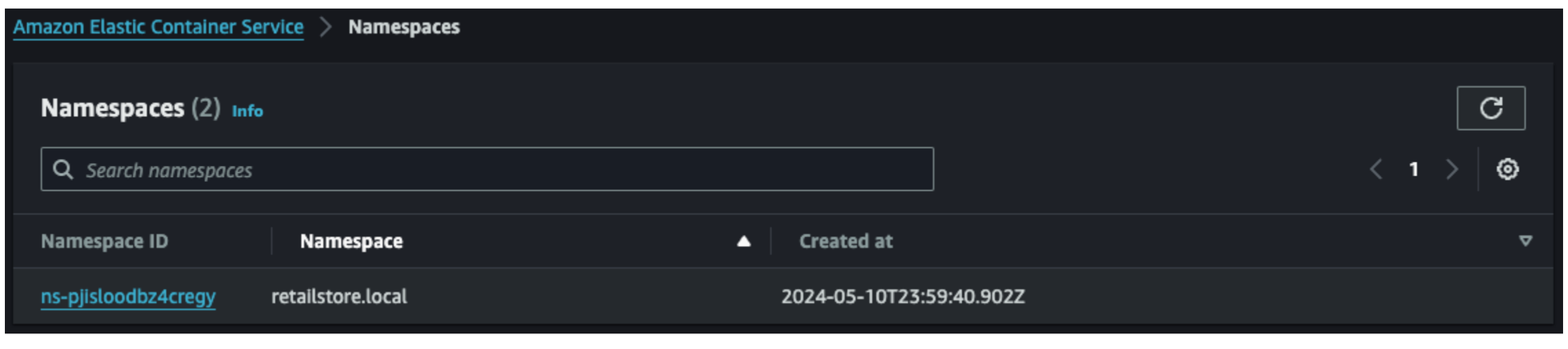
Since we also enabled Amazon GuardDuty for runtime monitoring in Amazon ECS, you can also see the `aws-guardduty-agent` container. More information on Amazon GuardDuty can be found here



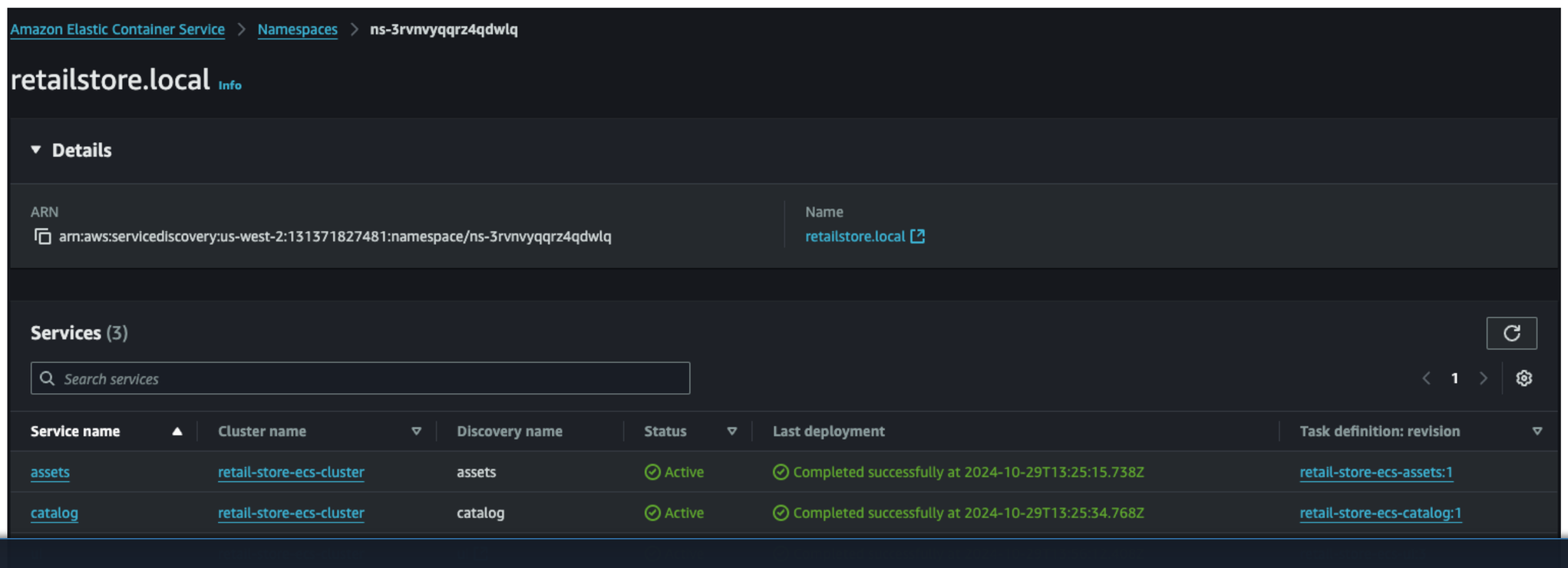
The Namespace

Service Connect uses AWS Cloud Map namespaces as a logical grouping of Amazon ECS tasks that communicate with one another. Each Amazon ECS service can belong to only one namespace. The services within a namespace can be spread across different Amazon ECS clusters within the same AWS Region in the same AWS account.

To review the Cluster Namespaces, open the Amazon ECS dashboard and select **Namespace** from the side navigation bar.



Select a namespace to review the details of the AWS Cloud Map configuration associated with each service. The **Discovery Name** represents the associated short name that can be used to connect to the service for services running in the same namespace (e.g., `http://assets`).



Metrics

Now let's review the metrics that Service Connect makes available.

Amazon ECS provides CloudWatch metrics you can use to monitor your resources. Review the full list of available metrics here.

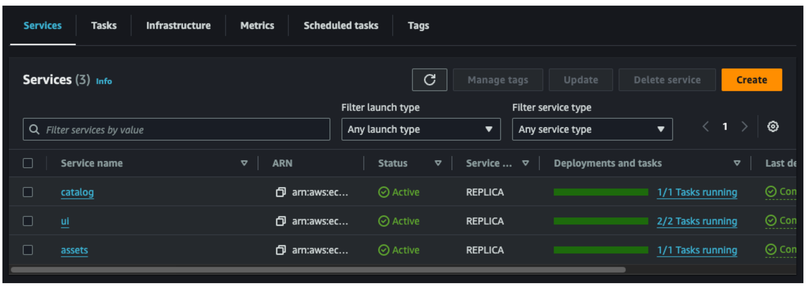
Execute the following command to generate some synthetic traffic:

```
export RETAIL_ALB=$(aws elbv2 describe-load-balancers --name retail-store-ecs-ui \
  --query 'LoadBalancers[0].DNSName' --output text)

hey -n 1000000 -c 1 -q 10 http://$RETAIL_ALB/home &
```

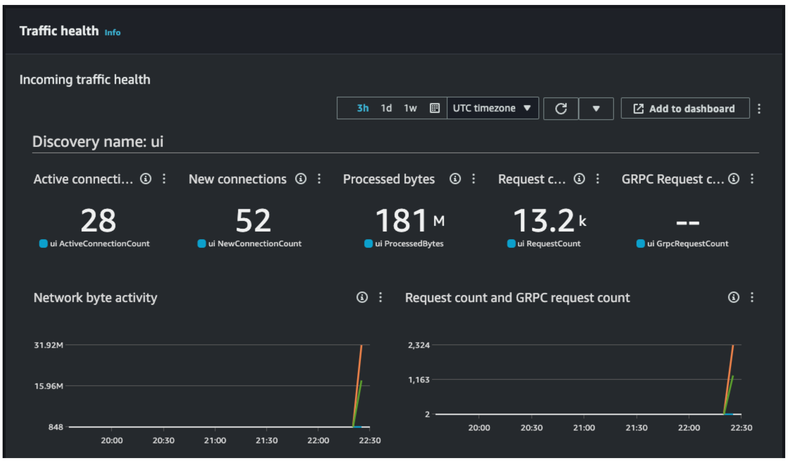
Review ECS Service Connect metric on the Traffic Health dashboard

Open the Amazon ECS Console, select the `retail-store-ecs-cluster`, and navigate to the Service tab.



Select the `ui` service and scroll down to the **Traffic Health** dashboard.

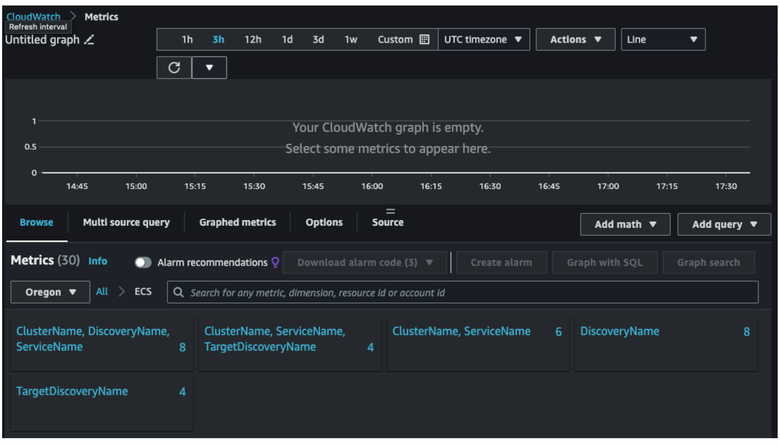
Amazon ECS sends metrics to CloudWatch every minute. When collecting metrics, Amazon ECS gathers multiple data points per minute. More information can be found here



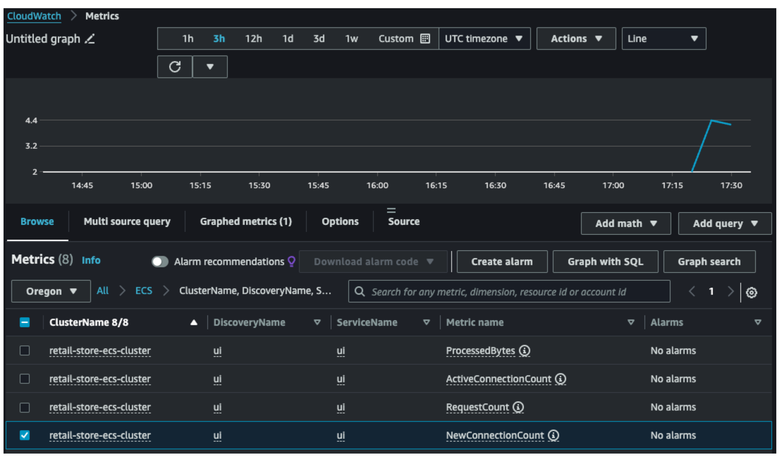
Amazon ECS configures tasks and containers so that applications only connect to the proxy if the application is connecting to endpoint names in the same namespace. All other traffic bypasses the proxy. This includes IP addresses in the same VPC, AWS service endpoints, and external traffic. More information can be found here. This explains why you won't find outgoing traffic for the `Assets` and `Catalog` services.

Review ECS Service Connect metric on Amazon CloudWatch

Open the Amazon CloudWatch metrics console:



Click on the **Cluster, DiscoveryName, ServiceName** metrics and search for `ui` to review the traffic metrics.



You can also explore **Cluster, ServiceName, TargetDiscoveryName** for additional metrics.

To stop the synthetic traffic generation, execute the following command:

Advanced ECS Service Connect

In this section, we will explore the ECS Service Connect capabilities by initiating an interactive session on the running containers to better understand the Service Connect configuration and capabilities.

Connect to the ECS Task

Execute the following command to select one of the running tasks with `enableExecuteCommand` enabled:

```
ECS_EXEC_TASK_ARN=$(aws ecs list-tasks --cluster retail-store-ecs-cluster \
    --service-name ui --query 'taskArns[]' --output text | \
    xargs -n1 aws ecs describe-tasks --cluster retail-store-ecs-cluster --tasks | \
    jq -r '.tasks[] | select(.enableExecuteCommand == true) | .taskArn' | \
    head -n 1)

echo $ECS_EXEC_TASK_ARN
```

This will output the ARN of the task:

```
arn:aws:ecs:us-west-2:XXXXXXXXXX:task/retail-store-ecs-cluster/0564778486a846599b8bd6b544e5f6eb
```

Now, start a `/bin/bash` interactive session in the running task:

```
if [ -z ${ECS_EXEC_TASK_ARN} ]; then echo "ECS_EXEC_TASK_ARN is not correctly configured!"; else
aws ecs execute-command --cluster retail-store-ecs-cluster \
    --task $ECS_EXEC_TASK_ARN \
    --container application \
    --interactive \
    --command "/bin/bash"
fi
```

Setup environment

In the Bash shell, install the `jq` command on the running container to format the JSON output on the command line:

```
yum install jq -y
```

Explore the host file

The `/etc/hosts` file provides the mapping of fully qualified domain names (FQDNs) to their respective IP addresses. Review the content of the `/etc/hosts` file by running the following command:

```
cat /etc/hosts
```

```
127.0.0.1 localhost
X.X.X.X ip-X-X-X-X.us-west-2.compute.internal
127.255.0.1 assets
2600:f0f0:0:0:0:0:0:1 assets
127.255.0.2 catalog
2600:f0f0:0:0:0:0:0:2 catalog
127.255.0.3 ui
2600:f0f0:0:0:0:0:0:3 ui
```

The file contains the three **Discovery names** set up in the ECS Namespace with three different local IPs `127.255.0.X` (IPv4 and IPv6). These local IPs all point to the same ECS local proxy, which will redirect the traffic to the appropriate ECS remote proxy bound to the specific service.

Test the connection to the Catalog service

Test the connection to the Catalog API from the running container:

```
curl http://catalog/catalogue/tags | jq
```

```
[
  {
    "name": "smart",
    "displayName": "Smart"
  },
  {
    "name": "dress",
    "displayName": "Dress"
  },
  {
    "name": "luxury",
    "displayName": "Luxury"
  },
  {
    "name": "casual",
    "displayName": "Casual"
  }
]
```

To terminate your `exec` session, run:

```
exit
```

Amazon ECS Service Connect with TLS

Important
You must have completed the following chapters as pre-requisites for this lab:

- Fundamentals
- ECS Service Connect

Amazon ECS Service Connect supports automatic traffic encryption using *Transport Layer Security (TLS)* certificates for Amazon ECS services. By configuring your Amazon ECS services to use an AWS Private Certificate Authority (AWS Private CA), Amazon ECS automatically provisions TLS certificates to encrypt traffic between your Amazon ECS Service Connect services. Amazon ECS handles the generation, rotation, and distribution of TLS certificates used for traffic encryption. You can find more information here.

In the following sections, you will:

- Deploy a new version of the `ui` service with TLS enabled, called the `ui-tls` service
- Enable TLS for the existing `Catalog` and `Assets` services
- Verify that TLS is enabled for Service Connect

Previous Next

Check Prerequisite

In this section, we will review and create the necessary AWS resources to set up ECS Service Connect with TLS, ensuring encryption for both inbound and outbound network traffic managed by ECS Service Connect. For more information, click here.

We will check each resource with the following steps:

- Review the existing AWS Private Certificate Authority.
- Review the existing Application Load Balancer Configuration.
- Create a new IAM Role required to issue certificates used by ECS Service Connect.

Review the existing AWS Private Certificate Authority

Let's review the configuration of the existing AWS Private Certificate Authority. We've created this in advance for a smoother hands-on experience.

```
CERTIFICATE_AUTH_ARN=$(aws acm-pca list-certificate-authorities \
    --query 'CertificateAuthorities[?Status==`ACTIVE` && CertificateAuthorityConfiguration.Subject.Organization==`ECS Immersion Day`].Arn' \
    --output text)

aws acm-pca describe-certificate-authority --certificate-authority-arn=$CERTIFICATE_AUTH_ARN
```

Expand for exploring private certificate authority detail

Review the existing Application Load Balancer Configuration

For convenience, we have pre-built resources related to the ALB (Application Load Balancer), including the listener, target group, and security group. By running the following command, we can review the details of the **ui-application-tls** Target Group associated with the HTTPS listener.

```
export UI_TARGET_GROUP_TLS_ARN=$(aws elbv2 describe-target-groups --names ui-application-tls \
 --query 'TargetGroups[0].TargetGroupArn' --output text)

aws elbv2 describe-target-groups --target-group-arns $UI_TARGET_GROUP_TLS_ARN
```

Expand for exploring target group detail

Now, you need to connect the Target Group to an HTTPS listener of the **retail-store-ecs-ui** Application Load Balancer (ALB). When creating the HTTPS Listener, since ECS Service Connect only supports TLS 1.3, you will need to specify the correct SSL Policy.

To create the HTTPS listener, run the following command:

```
RETAIL_ALB_ARN=$(aws elbv2 describe-load-balancers \
     --name retail-store-ecs-ui \
     --query 'LoadBalancers[0].LoadBalancerArn' \
     --output text)

aws elbv2 create-listener \
    --load-balancer-arn $RETAIL_ALB_ARN \
    --default-action '[{"Type": "forward", "TargetGroupArn": "'$UI_TARGET_GROUP_TLS_ARN'"}]' \
    --certificates CertificateArn=$ALB_CERTIFICATE_ARN \
    --protocol HTTPS \
    --port 443 \
    --ssl-policy ELBSecurityPolicy-TLS13-1-2-2021-06
```

Expand to explore listener details

Create a new IAM Role required to issue certificates used by ECS Service Connect

Additional IAM permissions are required to issue certificates. Amazon ECS provides a managed resource trust policy that outlines the set of permissions. For more information about this policy, see AmazonECSInfrastructureRolePolicyForServiceConnectTransportLayerSecurity.

```
cat << EOF > ecs-service-connect-certificate-policy.json
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "AllowAccessToECSForInfrastructureManagement",
            "Effect": "Allow",
            "Principal": {
                "Service": "ecs.amazonaws.com"
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
EOF
```

Run the following command to create a new **ecs-service-connect-certificate-role** role with the required managed policy:

```
aws iam create-role \
    --role-name ecs-service-connect-certificate-role \
    --assume-role-policy-document file://ecs-service-connect-certificate-policy.json \
    --query Role.Arn \
    --output text
```

```
aws iam attach-role-policy \
     --role-name ecs-service-connect-certificate-role \
     --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSInfrastructureRolePolicyForServiceConnectTransportLayerSecurity
```

Deploy the UI Service with TLS

In this section, we will deploy the UI service with TLS enabled ECS Service Connect and explore this service via HTTPS endpoint.

Deploy the UI service with TLS enabled in ECS Service Connect

We are now ready to set up the new **ui-tls** service with ECS Service Connect, ensuring TLS encryption is enabled.

```
export ECS_SERVICE_CONNECT_ROLE_ARN=$(aws iam get-role --role-name ecs-service-connect-certificate-role --query Role.Arn --output text)
```

Let's create the ECS service:

```
aws ecs create-service \
    --cluster retail-store-ecs-cluster \
    --service-name ui-tls \
    --task-definition retail-store-ecs-ui \
    --desired-count 1 \
    --enable-execute-command \
    --launch-type FARGATE \
    --load-balancers targetGroupArn=${UI_TARGET_GROUP_TLS_ARN},containerName=application,containerPort=8080 \
    --network-configuration "awsvpcConfiguration={subnets=[${PRIVATE_SUBNET1},${PRIVATE_SUBNET2}],securityGroups=[${UI_SG_ID}],assignPublicIp=DISABLED}" \
    --service-connect-configuration '{
        "enabled": true,
        "namespace": "retailstore.local",
        "services": [
            {
                "portName": "application",
                "discoveryName": "ui-tls",
                "clientAliases": [
                    {
                        "port": 80,
                        "dnsName": "ui-tls"
                    }
                ],
                "tls": {
                  "issuerCertificateAuthority": {
                    "awsPcaAuthorityArn": "'${CERTIFICATE_AUTH_ARN}'"
                  },
                  "roleArn": "'${ECS_SERVICE_CONNECT_ROLE_ARN}'"
                }
            }
        ]
    }'
```

You can wait for the service to stabilize using the AWS CLI (**~ 2 min**):

```
echo "Waiting for service to stabilize..."

aws ecs wait services-stable --cluster retail-store-ecs-cluster --services ui-tls
```

Review the new web application via the HTTPS endpoint

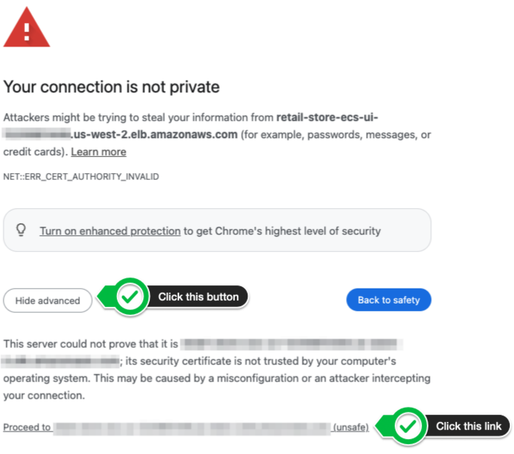
> ⚠ For this workshop, we used a self-signed certificate that will be considered invalid by your browser.

Once the new **UI-TLS** service has been deployed, we can review the running application over **HTTPS**.

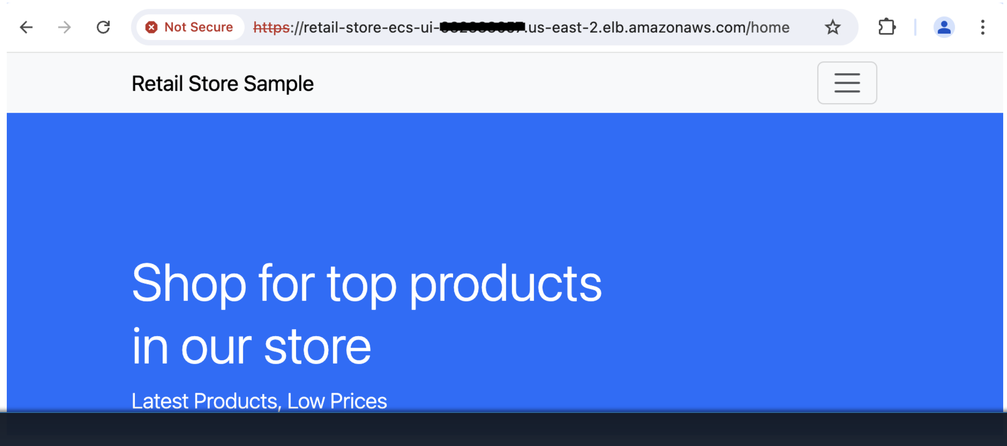
```
export RETAIL_ALB=$(aws elbv2 describe-load-balancers --name retail-store-ecs-ui \
 --query 'LoadBalancers[0].DNSName' --output text)

echo https://${RETAIL_ALB} ; echo
```

As shown in the image below, click the **Advanced** button and then click the link at the bottom stating `Proceed to retail-store-ecs-ui-XXXXXXXXXXX.us-west-2.elb.amazonaws.com (unsafe).`



After accessing the application, you should see the following page:



Deploy other services with TLS

In the previous section, we configured ECS Service Connect on the UI Service to encrypt communication between the Application Load Balancer and the UI Service. In this section, we will set up ECS Service Connect for the communication between **UI-TLS**, **Catalog**, and **Asset** services.

Set up TLS on ECS Service Connect for the Catalog Service

Update the **Catalog** Service to enable TLS for ECS Service Connect:

```
aws ecs update-service \
    --cluster retail-store-ecs-cluster \
    --service catalog \
    --task-definition retail-store-ecs-catalog \
    --force-new-deployment \
    --service-connect-configuration '{
        "enabled": true,
        "namespace": "retailstore.local",
        "services": [
            {
                "portName": "application",
                "discoveryName": "catalog",
                "clientAliases": [
                    {
                        "port": 80,
                        "dnsName": "catalog"
                    }
                ],
                "tls": {
                  "issuerCertificateAuthority": {
                    "awsPcaAuthorityArn": "'${CERTIFICATE_AUTH_ARN}'"
                  },
                  "roleArn": "'${ECS_SERVICE_CONNECT_ROLE_ARN}'"
                }
            }
        ]
    }'
```

Set up TLS on ECS Service Connect for the Assets Service

Update the **Assets** Service to enable TLS for ECS Service Connect (**~ 5 min**):

```
aws ecs update-service \
    --cluster retail-store-ecs-cluster \
    --service assets \
    --task-definition retail-store-ecs-assets \
    --force-new-deployment \
    --service-connect-configuration '{
        "enabled": true,
        "namespace": "retailstore.local",
        "services": [
            {
                "portName": "application",
                "discoveryName": "assets",
                "clientAliases": [
                    {
                        "port": 80,
                        "dnsName": "assets"
                    }
                ],
                "tls": {
                  "issuerCertificateAuthority": {
                    "awsPcaAuthorityArn": "'${CERTIFICATE_AUTH_ARN}'"
                  },
                  "roleArn": "'${ECS_SERVICE_CONNECT_ROLE_ARN}'"
                }
            }
        ]
    }'
```

```
echo "Waiting for service to stabilize..."

aws ecs wait services-stable --cluster retail-store-ecs-cluster --services catalog
aws ecs wait services-stable --cluster retail-store-ecs-cluster --services assets
```

Now, all traffic between the **UI-TLS**, **Catalog**, and **Assets** services is encrypted.

ECS Service Connect TLS Verification

Now that we have completed the ECS Service Connect setup, we will verify our configuration to ensure that ECS Service Connect TLS is correctly enabled. In this section, we will perform the following steps:

- Retrieve the private IP of tasks
- Connect to a running task with `enableExecuteCommand` enabled
- Install the `openssl` command on the running container
- Verify the TLS certificate

Retrieve the private IP of tasks

Calling the DNS name directly does not reveal the certificate.

Run the following command to retrieve the private IP address of a running **UI-TLS** task and save the result in an environment variable:

```
task_arn=$(aws ecs list-tasks --cluster retail-store-ecs-cluster \
    --service-name ui-tls \
    --query 'taskArns[0]' --output text)

UI_TLS_PRIVATE_IP=$(aws ecs describe-tasks --cluster retail-store-ecs-cluster --tasks $task_arn \
    | jq -r '.tasks[].containers[] | select(.name == "application") | .networkInterfaces[0].privateIpv4Address')
```

You can verify the IP address with the following command:

```
echo ${UI_TLS_PRIVATE_IP}
```

Connect to a running task with `enableExecuteCommand` enabled

Run the following command to select one of the **UI** tasks with `enableExecuteCommand` enabled:

```
export ECS_EXEC_TASK_ARN=$(aws ecs list-tasks --cluster retail-store-ecs-cluster \
    --service-name ui --query 'taskArns[0]' --output text | \
    xargs -n1 aws ecs describe-tasks --cluster retail-store-ecs-cluster --tasks | \
    jq -r '.tasks[] | select(.enableExecuteCommand == true) | .taskArn' | \
    head -n 1)

echo ${ECS_EXEC_TASK_ARN}
```

Copy the environment variable to the running task:

```
aws ecs execute-command --cluster retail-store-ecs-cluster \
    --task $ECS_EXEC_TASK_ARN \
    --container application \
    --interactive \
    --command "/bin/bash -c 'echo UI_TLS_PRIVATE_IP=$(echo $UI_TLS_PRIVATE_IP) > /app/private_ip.env'"
```

Start an interactive `/bin/bash` session in the running task:

```
aws ecs execute-command --cluster retail-store-ecs-cluster \
    --task $ECS_EXEC_TASK_ARN \
    --container application \
    --interactive \
    --command "/bin/bash"
```

Set up the environment

Load the environment variable into the current session:

```
source /app/private_ip.env
```

Install the `openssl` command on the running container to retrieve and view the existing certificate:

```
yum install openssl -y
```

Verify the TLS Certificate on the `ui-tls` task

Run the following command to retrieve and review the certificate used by the `ui-tls` task with ECS Service Connect TLS enabled:

```
openssl s_client -connect $UI_TLS_PRIVATE_IP:8080 < /dev/null 2> /dev/null \
    | openssl x509 -noout -text
```

Below is the output showing all details of the existing certificate:

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = US, O = ECS Immersion Day, OU = Workshop
        Validity
            Not Before: Aug 18 13:55:27 2024 GMT
            Not After : Aug 25 14:55:27 2024 GMT
        Subject: OU = Workshop, O = ECS Immersion Day, C = US
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
                Public-Key: (256 bit)
                pub:
                    04:85:6e:c0:43:44:f9:e9:7d:ba:ec:6c:fe:0e:37:
                    ...
                    53:82:c2:a2:20
                ASN1 OID: prime256v1
                NIST CURVE: P-256
        X509v3 extensions:
            X509v3 Subject Alternative Name:
                DNS:ui-tls.retailstore.local
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 Authority Key Identifier:
                11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11
            X509v3 Subject Key Identifier:
                11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11:11
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication, TLS Web Client Authentication
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        a7:dc:2e:28:9b:ae:2d:93:de:05:93:56:e3:c8:81:50:fe:3f:
        ...
        70:92:7b:c1:b7:16:9b:8a:16:61:c1:aa:ab:9e:de:d3:e4:e7:
        cb:05:86:00
```

To terminate your exec session, run:

```
exit
```